A.Introduction

Project overview : project management like jira
Tech stack : Angular frontend
Authentication flow
User register first of all ok hes now logged in cookies and access tokens and refresh are made

Post :/api/v1/auth/register

```
export const register = catchAsync(async (req, res, next) => {
    const { username, email, password, confirmPassword } = req.body;

    if (!username || !email || !password || !confirmPassword) {
        return next(new AppError(400, "All fields are required"));
    }

    if (password !== confirmPassword) {
        return next(new AppError(400, "Passwords do not match"));
    }

    const existing = await User.findOne({ email: email });
    if (existing) {
        if (existing.isActive !== false) {
            return next(new AppError(409, "Email already exists"));
        }

        await User.findByIdAndDelete(existing._id);
    }
    const existingName = await User.findOne({ username: username });
    if(existingName){
        return next(new AppError(409, "sorry username already exists"));

    }

    const newUser = await User.create(req.body);


    createCookie(res, newUser._id, "accessToken",
signJwtAccessToken(newUser._id));
```

```
   const refreshToken = signJwtRefreshToken(newUser._id);

   await redis.set(`refreshToken:${newUser._id}`, refreshToken, {
       EX: ms(process.env.JWT_REFRESH_EXPIRATION) / 1000,
   });
    createCookie(res, newUser._id, "refreshToken", refreshToken);

   const responseUser = {
 _id: newUser._id,
 username: newUser.username,
 email: newUser.email,
 avatar: newUser.avatar,
 role: newUser.role,
 isVerified: newUser.isVerified,
 isActive: newUser.isActive,
 createdAt: newUser.createdAt,
};
   const userEmail= new Email("hello thank you for registering with us
",responseUser);
   await userEmail.send("welcome to project management app");
    res.status(201)
      .json(ApiResponse.success("User registered
successfully",responseUser));
});
```

User can log in forgot password and reset password
Post :api/v1/users/forgot-password
Post :api/v1/users/reset-password

```
export const updatePassword = catchAsync(async (req,res,next)=>{
 const {oldPassword,newPassword,confirmNewPassword} = req.body;
 const userId = req.user.id;
 const changed = await
updateUserPassword(userId,oldPassword,newPassword,confirmNewPassword);
 if(!changed){
   return next(new AppError(400, 'sorry problem changing password'));
 }
return res.status(201).json(ApiResponse.success('password updated
successfully'));
```

```
});
export const forgotPassword = catchAsync(async (req,res,next)=>{
const userEmail = req.body.email;
const user = await User.findOne({email:userEmail});
if(!user){
 return res
   .status(200)
   .json(ApiResponse.success("If an account exists, a reset link has been
sent."));}
const resetToken = user.createResetToken();
await user.save({validateBeforeSave:false});
let resetURL;
if(process.env.ENVIRONMENT === "development"){
resetURL=
`${req.protocol}://${req.get('host')}/api/v1/users/reset-password?token=${
resetToken}`;

}else if(process.env.ENVIRONMENT === "production"){
  resetURL =
`${process.env.FRONTEND_URL}/reset-password?token=${resetToken}`;

}
const email = new Email(`here is your reset password link ${resetURL}
valid for 10 minutes only`,user) ;
try{
await email.send("reset password link");}
catch(err){
      user.passwordResetToken = undefined;
   user.passwordResetExpires = undefined;
   await user.save({ validateBeforeSave: false });

   return next(new AppError(500, "Failed to send email"));

}
return res.status(200).json(ApiResponse.success('reset password link sent
to your email'));
});
export const resetPassword = catchAsync(async(req,res,next)=>{
const {password,confirmPassword} = req.body;
const {token}  = req.query;
```

```
if (!token) {
 return next(new AppError(400, "Reset token is required"));
}
 const hashedToken =
crypto.createHash('sha256').update(token).digest('hex');
if(password!==confirmPassword){
        return next(new AppError(400, "Failed to reset password Passwords
do not match"));

}
 const user = await
User.findOne({passwordResetToken:hashedToken,passwordResetExpires:{$gt:Dat
e.now()}});
 if(!user){
        return next(new AppError(400, "Failed to reset password Invalid or
expired reset token"));
 }
user.passwordResetToken = undefined;
user.passwordResetExpires = undefined;
user.password = password;
await user.save();
res.status(200).json(ApiResponse.success('password reset successfully'));


});
```

What you the front end need to know

Middleware protect `export const protect =catchAsync(async (req,res,next)=>{`

```
const accessToken = req.cookies?.accessToken;
 if(!accessToken){
   return next(new AppError(401,"please Log in first"));
 }
let decoded;
 try{
 const accessSecret = process.env.JWT_ACCESS_SECRET ||
process.env.JWT_SECRET;
 decoded = jwt.verify(accessToken, accessSecret);
 }catch(err){
```

```
 return next( new AppError(401,"your session has expired please log in
again"));
 }
const myUser  = await User.findById(decoded.id);
 if(!myUser){
        return next(new AppError(401,"The user belonging to this token no
longer exists."));
 }

req.user= myUser;
next();
});
```

Checksif user is logged in he check accestoken is available fine continue
If Expired you received 401 then hit /api/v1/auth/refresh

```
export const refresh = catchAsync(async (req,res,next)=>{
const  refreshToken = req.cookies.refreshToken;
console.log(refreshToken);
if(!refreshToken){
return next(new AppError(401,"please log in first"));
}
      let decoded;
   try {
       const refreshSecret = process.env.JWT_REFRESH_SECRET;
       decoded = jwt.verify(refreshToken, refreshSecret);
   } catch (err) {
       return next(new AppError(401, "Your session has expired. Please log
in again."));
   }
     const user = await User.findById(decoded.id);
   if(!user){
       return next(new AppError(401, "The user belonging to this token no
longer exists."));
   }
   createCookie(res, user._id,
"accessToken"),signJwtAccessToken(user._id);

   res.status(200).json(ApiResponse.success("Token renewed"));
```

```
});
```

it checks the refresh token if its non expired then continue if its expired hit
Post :api/v1/auth/login

```
export const login = catchAsync(async (req,res,next)=>{
   const {email,password}=req.body;
   if(!email || !password){
       return next(new AppError(400,"all fields are required"));
   }
const myUser = await User.findOne({email:email})
;
   if(! myUser || !(await Bcrypt.compare(password,myUser.password))){
       return next(new AppError(401,"sorry invalid credentiels"));
   }

   createCookie(res, myUser._id, "accessToken",
signJwtAccessToken(myUser._id));

   const refreshToken = signJwtRefreshToken(myUser._id);

   await redis.set(`refreshToken:${myUser._id}`, refreshToken, {
       EX: ms(process.env.JWT_REFRESH_EXPIRATION) / 1000,
   });
    createCookie(res, myUser._id, "refreshToken", refreshToken);


       res.status(200).json(ApiResponse.success("User logged in
successfully"));

});
```

Base API URL: port 8000 for now and api/v1/

B. Pages Overview

1. Authentication

Login

Endpoint

- `POST /auth/login`

Register

Endpoint

- `POST /auth/register`

Logout

Endpoint

- `POST /auth/logout`

Refresh Token

Endpoint

- `POST /auth/refresh`

Forgot Password

Endpoint

- `POST /auth/forgot-password`

Reset Password

Endpoint

- `POST /auth/reset-password`

---

2. Dashboard (Home)

Uses

- `GET /users/me`

Shows

- Welcome message
- Projects count
- Tasks count
- Completed tasks
- Notifications count
- Recent activity (optional)

---

3. Projects

Authorization Middleware

```
export const authorizeProjectTo = (...roles) => {

 return async (req, res, next) => {

   const { projectId } = req.params;



   const project = await Project.findById(projectId);
```

```
    if (!project) {
      return next(new AppError(404, "Project not found"));
    }

    const member = project.members.find(
      (m) => m.user.toString() === req.user.id
    );

    if (!member) {
      return next(new AppError(403, "You are not a member of this
project"));
    }

    if (!roles.includes(member.role)) {
      return next(new AppError(403, "You are not authorized to perform this
action"));
    }

    next();
  };
};
```

Use this middleware for all project-related endpoints that require project membership or specific project roles.

Uses

- `GET /projects`
- `POST /projects`
- `GET /projects/:projectId`
- `PATCH /projects/:projectId`
- `DELETE /projects/:projectId`
- `GET /projects/:projectId/activity`

Shows

- Project cards
- Create Project button
- Search
- Pagination

---

4. Project Details

Uses

- `GET /projects/:projectId`
- `PATCH /projects/:projectId`
- `DELETE /projects/:projectId`
- `GET /projects/:projectId/activity`

Contains

- Project information
- Members
- Tasks
- Activity Feed (Project Logs)

The activity feed displays:

- User Avatar
- Username
- Action performed
- Related entity (Task, Comment, Attachment, Invitation, Member)
- Timestamp

Example log entries:

- John created task "Authentication"
- Sarah changed task status to Done
- Ahmed uploaded "design.pdf"
- Mohamed commented on "Dashboard"
- Omar accepted the invitation

---

5. Members

Uses

- `GET /projects/:projectId/members`
- `PATCH /projects/:projectId/members/:userId/role`

- `DELETE /projects/:projectId/members/:userId`

Shows

- Members list
- Roles
- Remove member
- Change role

---

6. Invitations

Authorization Middleware

`authorizeProjectTo('ADMIN', 'OWNER')`

Required for invitation management endpoints.

Uses

- `GET /invitations`
- `POST /invitations/:projectId`
- `PATCH /invitations/:projectId/:invitationId`
- `DELETE /invitations/:projectId/:invitationId`
- `PATCH /invitations/:invitationId`

Contains

- Pending invitations
- Invite member modal
- Accept invitation
- Reject invitation
- Cancel invitation

---

7. Tasks (Inside Project Details)

Authorization Middleware

```
export const authorizeTaskTo = (...roles) => {

 return async (req, res, next) => {

   const { taskId } = req.params;



   const task = await Task.findById(taskId);



   if (!task) {

     return next(new AppError(404, 'Task not found'));

   }



   const project = await Project.findById(task.project);



   if (!project) {

     return next(new AppError(404, 'Project not found'));

   }
```

```
    const member = project.members.find(
      (m) => m.user.toString() === req.user.id
    );

    if (!member) {
      return next(new AppError(403, 'You are not a member of this
project'));
    }

    if (roles.length && !roles.includes(member.role)) {
      return next(new AppError(403, 'You are not authorized to perform this
action'));
    }

    req.task = task;
    req.project = project;
    next();
  };
};
```

Use this middleware for all task-related endpoints.

Uses

- `GET /tasks`
- `GET /projects/:projectId/tasks`
- `POST /projects/:projectId/tasks`
- `GET /tasks/:taskId`
- `PATCH /tasks/:taskId`
- `DELETE /tasks/:taskId`

Shows

- Kanban board
- Task cards
- Create task
- Edit task
- Delete task

---

8. Task Details

Uses

- `GET /tasks/:taskId`

Contains

- Description
- Assignee
- Priority
- Due date
- Status

- Comments
- Attachments

---

9. Comments

Uses

- `POST /tasks/:taskId/comments`
- `DELETE /comments/:commentId`

Contains

- Comment list
- Add comment
- Delete comment

> Comments are displayed as part of the Task Details page.

---

10. Attachments

Uses

- `POST /tasks/:taskId/attachments`
- `DELETE /tasks/:taskId/attachments/:attachmentId`

Contains

- Upload attachment

- Download attachment
- Delete attachment

  Attachments are displayed as part of the Task Details page.

---

11. Notifications

Uses

- `GET /notifications`
- `PATCH /notifications/read`

Contains

- Notification list
- Mark all as read

---

12. Profile

Uses

- `GET /users/me`
- `PATCH /users/me`
- `PATCH /users/password`
- `DELETE /users/me`

Contains

- Avatar

- Username
- Email
- Change password
- Delete account

---

13. Admin Panel

Authorization Middleware

`authorizedTo('admin')`

```
export  const authorizedTo = (...roles)=>{

 return (req, res, next) => {

   if (!roles.includes(req.user.role)) {

     return next(new AppError(403, 'You are not authorized to perform this
action'));

   }

   next();

 };

};
```

Required for all admin endpoints.

Uses

- `GET /users`
- `GET /users/:id`

- `PATCH /users/:id`
- `DELETE /users/:id`

Shows

- Users table
- Search
- Filters
- User management

C.code

Main flow route —-> controller —---> service

Herethere is no service for auth stuff only

```
router.post("/login",login);
router.post("/register",register);
router.post("/logout",logout);
router.post("/refresh",refresh);
const signJwtAccessToken = (userId) => {
   const accessSecret =  process.env.JWT_SECRET;
   const accessExpiresIn = process.env.JWT_ACCESS_EXPIRATION;
   return jwt.sign({ id: userId }, accessSecret, {
       expiresIn: accessExpiresIn,
   });
};

const signJwtRefreshToken = (userId) => {
   const refreshSecret = process.env.JWT_REFRESH_SECRET;
   const refreshExpiresIn = process.env.JWT_REFRESH_EXPIRATION ;
```

```
    return jwt.sign({ id: userId }, refreshSecret, {
        expiresIn: refreshExpiresIn,
    });
};

const createCookie = (res, userId, type,token) => {
    if (type === "accessToken") {
        const accessMaxAge =ms(process.env.JWT_ACCESS_EXPIRATION);
        return res.cookie("accessToken", token, {
            httpOnly: true,
            secure: process.env.NODE_ENV === "production",
            maxAge: accessMaxAge,
        });
    } else if (type === "refreshToken") {
        const refreshMaxAge =  ms(process.env.JWT_REFRESH_EXPIRATION);
        return res.cookie("refreshToken", token, {
            httpOnly: true,
            secure: process.env.NODE_ENV === "production",
            maxAge: refreshMaxAge,
        });
    }
};

export const register = catchAsync(async (req, res, next) => {
    const { username, email, password, confirmPassword } = req.body;

    if (!username || !email || !password || !confirmPassword) {
        return next(new AppError(400, "All fields are required"));
    }

    if (password !== confirmPassword) {
        return next(new AppError(400, "Passwords do not match"));
    }

    const existing = await User.findOne({ email: email });
    if (existing) {
        if (existing.isActive !== false) {
            return next(new AppError(409, "Email already exists"));
        }
```

```
        await User.findByIdAndDelete(existing._id);
    }
    const existingName = await User.findOne({ username: username });
    if(existingName){
        return next(new AppError(409, "sorry username already exists"));

    }

    const newUser = await User.create(req.body);


    createCookie(res, newUser._id, "accessToken",
signJwtAccessToken(newUser._id));

    const refreshToken = signJwtRefreshToken(newUser._id);

    await redis.set(`refreshToken:${newUser._id}`, refreshToken, {
        EX: ms(process.env.JWT_REFRESH_EXPIRATION) / 1000,
    });
     createCookie(res, newUser._id, "refreshToken", refreshToken);

    const responseUser = {
 _id: newUser._id,
 username: newUser.username,
 email: newUser.email,
 avatar: newUser.avatar,
 role: newUser.role,
 isVerified: newUser.isVerified,
 isActive: newUser.isActive,
 createdAt: newUser.createdAt,
};
    const userEmail= new Email("hello thank you for registering with us
",responseUser);
    await userEmail.send("welcome to project management app");
     res.status(201)
       .json(ApiResponse.success("User registered
successfully",responseUser));
});
```

```
export const login = catchAsync(async (req,res,next)=>{
   const {email,password}=req.body;
   if(!email || !password){
       return next(new AppError(400,"all fields are required"));
   }
const myUser = await User.findOne({email:email})
;
   if(! myUser || !(await Bcrypt.compare(password,myUser.password))){
       return next(new AppError(401,"sorry invalid credentiels"));
   }

   createCookie(res, myUser._id, "accessToken",
signJwtAccessToken(myUser._id));

   const refreshToken = signJwtRefreshToken(myUser._id);

   await redis.set(`refreshToken:${myUser._id}`, refreshToken, {
       EX: ms(process.env.JWT_REFRESH_EXPIRATION) / 1000,
   });
    createCookie(res, myUser._id, "refreshToken", refreshToken);


       res.status(200).json(ApiResponse.success("User logged in
successfully"));

});


export const logout = catchAsync(async (req, res, next) => {
try{
   res.clearCookie("accessToken", {
       httpOnly: true,
       secure: process.env.NODE_ENV === "production",
   });

   res.clearCookie("refreshToken", {
       httpOnly: true,
       secure: process.env.NODE_ENV === "production",
   });
```

```
   await redis.del(`refreshToken:${req.user._id}`);
  }catch(err){
   return next(new AppError(500,"sorry problem logging out"));
  }
   res.status(200).json(ApiResponse.success("Logged out successfully"));
});

export const refresh = catchAsync(async (req,res,next)=>{
const  refreshToken = req.cookies.refreshToken;
console.log(refreshToken);
if(!refreshToken){
return next(new AppError(401,"please log in first"));
}
      let decoded;
   try {
       const refreshSecret = process.env.JWT_REFRESH_SECRET;
       decoded = jwt.verify(refreshToken, refreshSecret);
   } catch (err) {
       return next(new AppError(401, "Your session has expired. Please log
in again."));
   }
     const user = await User.findById(decoded.id);
   if(!user){
       return next(new AppError(401, "The user belonging to this token no
longer exists."));
   }
   createCookie(res, user._id,
"accessToken"),signJwtAccessToken(user._id);

   res.status(200).json(ApiResponse.success("Token renewed"));


});
```

Users

Model

```
const userSchema = new mongoose.Schema(
 {

   username: {
     type: String,
     required: [true, "Username is required"],
     unique: true,
     trim: true,
     minlength: [5, "Username must be at least 5 characters"],
     maxlength: [50, "Username cannot exceed 50 characters"],
     match: [/^[A-Za-z0-9_]+$/,"Username can only contain letters,
numbers, and underscores",
   ],
    },
   email: {
     type: String,
     required: [true, "Email is required"],
     unique: true,
     trim: true,
     lowercase: true,
     match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, "Please provide a valid email
address"],
   },
   password: {
     type: String,
     required: [true, "Password is required"],
     minlength: [8, "Password must be at least 8 characters"],
   },
       avatar: {
     type: String,
     default: "",
   },avatarPublicId:
   {
      type: String,
     default: "",
   }
   ,
   role: {
```

```
    type: String,
    enum: ["user", "admin"],
    default: "user",
  },
  isVerified: {
    type: Boolean,
    default: false,
  },
  isActive:{
    type:Boolean,
    default:true
  },
  isOnline: {
    type: Boolean,
    default: false,
  },
  lastSeen: {
    type: Date,
    default: null,
  },
  createdAt:{
    type:Date,
    default : new Date()
  } ,
  passwordResetToken: String,
  passwordResetExpires: Date,

 },
 { timestamps: true }
);

userSchema.pre('save',async function(next) {
 if (!this.isModified('password')) return next();
 const salt = await Bcrypt.genSalt(10);
 this.password = await Bcrypt.hash(this.password, salt);
 next();
});

userSchema.methods.comparePassword = async function (candidatePassword) {
 try {
```

```
    // 'this.password' refers to the hashed password of the current user
document
    return await Bcrypt.compare(candidatePassword, this.password);
  } catch (error) {
    throw new Error(error);
  }
};



userSchema.methods.createResetToken = function () {
 const resetToken = crypto.randomBytes(32).toString("hex");

 this.passwordResetToken = crypto
    .createHash("sha256")
    .update(resetToken)
    .digest("hex");

 this.passwordResetExpires = Date.now() + 10 * 60 * 1000;

 return resetToken;
};
```

```
router.route('/forgot-password').post(forgotPassword);
router.route('/reset-password').post(resetPassword);
router.route('/').get(protect, authorizedTo('admin') ,getAllUsers);

router.route('/me').get(protect, getMe).patch(protect,
upload.single("avatar"),updateMe).delete(protect, deleteMe);
router.route('/password').patch(protect, updatePassword);


router.route('/:id').get(protect, authorizedTo('admin'),
getUserById).patch(protect, authorizedTo('admin'),
updateUserById).delete(protect, authorizedTo('admin'), deleteUserById);
```

```
export const getMe = catchAsync(async (req, res, next) => {
   const userId = req.user.id;
   let result;

   try {
       result = await userData(userId);
   } catch (err) {
       return next(new AppError(500, `Sorry, a problem occurred:
${err.message}`));
   }

   if (result === null || !result.user) {
       return next(new AppError(404, "User not found"));
   }

   res.status(200).json(
       ApiResponse.success(
           "User fetched successfully",
           result.user,
           null,
           result.summary
       )
   );
});

export const deleteMe = catchAsync(async (req, res, next) => {
 const userId = req.user.id;
 const done = await deleteUserByIdService(userId);
 if (!done) {
   return next(new AppError(404, 'sorry a problem occured'));
 }
 await Promise.all([
   Project.updateMany(
       {},
       {
           $pull: {
               members: { user: userId}
           }
```

```
        }
    ),

    Invitation.deleteMany({
        $or: [
            { sender: userId },
            { receiver: userId }
        ]
    }),

    Task.updateMany(
        { assignee: userId},
        {
            $unset: { assignee: "" }
        }
    )
]);


    res.clearCookie("accessToken", {
        httpOnly: true,
        secure: process.env.NODE_ENV === "production",
    });

    res.clearCookie("refreshToken", {
        httpOnly: true,
        secure: process.env.NODE_ENV === "production",
    });
    await redis.del(`refreshToken:${userId}`);
    await redis.del(`user:${userId}:profile`);

 res.status(200).json(ApiResponse.success('User deleted successfully',
null));
});

export const updateMe = catchAsync(async (req, res, next) => {
 const userId = req.user.id;
 const data = req.body;
 if (  Object.keys(req.body).length === 0 &&
 !req.file) {
```

```
   return next(new AppError(404, 'you must provide data to update'));
 }
  if (data.password) {
   return next(new AppError(400, 'Password cannot be updated here'));
 }

 const updatedUser = await updateUser(userId, data,req.file);

 if (!updatedUser) {
   return next(new AppError(404, 'sorry a problem occured'));
 }

 res.status(201).json(ApiResponse.success('User updated successfully',
updatedUser));
});

export const getAllUsers = catchAsync(async (req, res,next) => {
try{
 const result = await getAllUsersService(req.query);

res.status(200).json(
 ApiResponse.success(
   "Users fetched successfully",
   result.data,
   result.meta,
   result.summary
 )
);}
catch(err){
 next(new AppError(500,`sorry a problem occured: ${err.message}`));
}

});

export const createUser = catchAsync(async (req, res, next) => {
 const user = await createUserService(req.body);
 if (!user) {
   return next(new AppError(400, 'User could not be created'));
 }
```

```
 res.status(201).json(ApiResponse.success('User created successfully',
user));
});

export const getUserById = catchAsync(async (req, res, next) => {
 const user = await getUserByIdService(req.params.id);
 if (!user) {
   return next(new AppError(404, 'User not found'));
 }

 res.status(200).json(ApiResponse.success('User fetched successfully',
user));
});

export const updateUserById = catchAsync(async (req, res, next) => {
 const user = await updateUserByIdService(req.params.id, req.body);
 if (!user) {
   return next(new AppError(404, 'User not found'));
 }

 res.status(200).json(ApiResponse.success('User updated successfully',
user));
});

export const deleteUserById = catchAsync(async (req, res, next) => {
 const user = await deleteUserByIdService(req.params.id);

 if (!user) {
   return next(new AppError(404, 'User not found'));
 }
       res.clearCookie("accessToken", {
       httpOnly: true,
       secure: process.env.NODE_ENV === "production",
   });

   res.clearCookie("refreshToken", {
       httpOnly: true,
       secure: process.env.NODE_ENV === "production",
   });
   await redis.del(`refreshToken:${id}`);
```

```
 res.status(200).json(ApiResponse.success('User deleted successfully',
user));
});
export const updatePassword = catchAsync(async (req,res,next)=>{
 const {oldPassword,newPassword,confirmNewPassword} = req.body;
 const userId = req.user.id;
 const changed = await
updateUserPassword(userId,oldPassword,newPassword,confirmNewPassword);
 if(!changed){
   return next(new AppError(400, 'sorry problem changing password'));
 }
return res.status(201).json(ApiResponse.success('password updated
successfully'));
});
export const forgotPassword = catchAsync(async (req,res,next)=>{
const userEmail = req.body.email;
const user = await User.findOne({email:userEmail});
if(!user){
 return res
   .status(200)
   .json(ApiResponse.success("If an account exists, a reset link has been
sent."));}
const resetToken = user.createResetToken();
await user.save({validateBeforeSave:false});
let resetURL;
if(process.env.ENVIRONMENT === "development"){
resetURL=
`${req.protocol}://${req.get('host')}/api/v1/users/reset-password?token=${
resetToken}`;

}else if(process.env.ENVIRONMENT === "production"){
  resetURL =
`${process.env.FRONTEND_URL}/reset-password?token=${resetToken}`;

}
const email = new Email(`here is your reset password link ${resetURL}
valid for 10 minutes only`,user) ;
try{
await email.send("reset password link");}
catch(err){
```

```
      user.passwordResetToken = undefined;
   user.passwordResetExpires = undefined;
   await user.save({ validateBeforeSave: false });

   return next(new AppError(500, "Failed to send email"));

}
return res.status(200).json(ApiResponse.success('reset password link sent
to your email'));
});
export const resetPassword = catchAsync(async(req,res,next)=>{
const {password,confirmPassword} = req.body;
const {token}  = req.query;
if (!token) {
 return next(new AppError(400, "Reset token is required"));
}
 const hashedToken =
crypto.createHash('sha256').update(token).digest('hex');
if(password!==confirmPassword){
        return next(new AppError(400, "Failed to reset password Passwords
do not match"));

}
 const user = await
User.findOne({passwordResetToken:hashedToken,passwordResetExpires:{$gt:Dat
e.now()}});
 if(!user){
        return next(new AppError(400, "Failed to reset password Invalid or
expired reset token"));
 }
user.passwordResetToken = undefined;
user.passwordResetExpires = undefined;
user.password = password;
await user.save();
res.status(200).json(ApiResponse.success('password reset successfully'));


});
```

```
export const userData = async (id) => {
   const key = `user:${id}:profile`;

   // 1. Check Redis first
   const cached = await redis.get(key);

   if (cached) {
       console.log('Redis HIT');
       return JSON.parse(cached);
   }

   console.log('Redis MISS');

   // 2. Fetch user
   const user = await User.findById(id)
       .select('-password')
       .lean();

   if (!user) {
       return null;
   }

   // 3. Calculate dashboard stats in parallel
   const [
       projectCount,
       taskCount,
       completedTaskCount,
       unreadNotifications,
       pendingInvitations
   ] = await Promise.all([
       Project.countDocuments({ 'members.user': id }),
       Task.countDocuments({ assignee: id }),
       Task.countDocuments({ assignee: id, status: 'DONE' }),
       Notification.countDocuments({ receiver: id, isRead: false }),
       Invitation.countDocuments({ receiver: id, status: 'PENDING' })
   ]);

   const summary = {
```

```
        projectCount,
        taskCount,
        completedTaskCount,
        unreadNotifications,
        pendingInvitations
    };

    const result = { user, summary };

    // 4. Cache the whole result for 5 minutes
    await redis.set(
        key,
        JSON.stringify(result),
        {
            EX: 60 * 5
        }
    );

    return result;
};
// export const deleteUser = async (id) => {

//   return await User.findByIdAndUpdate(id, { isActive: false }, { new:
true }).select('-password').lean().exec();
// };

export const updateUser = async (id, data, file) => {
 const user = await User.findById(id);
 await redis.del(`user:${id}:profile`);
 if (!user) {
   throw new AppError(404, 'user not found');
 }

 if (file) {
const allowedTypes = ['image/jpeg', 'image/png', 'image/jpg'];

if (!allowedTypes.includes(file.mimetype)) {
 throw new AppError(400, 'Only JPG, JPEG, and PNG images are allowed');
}
    if (user.avatarPublicId) {
```

```
      await imagekit.deleteFile(user.avatarPublicId);
    }

    // This calls your working ImageKit helper
    try{
    const uploaded = await uploadToCloudinary(file);

    user.avatar = uploaded.secure_url;
    user.avatarPublicId = uploaded.public_id;
  }
    catch(err){
      throw new AppError(500,`file upload failed: ${err.message}`);
    }
  }

  const allowedFields = ['username', 'email'];

  allowedFields.forEach((field) => {
    if (data[field] !== undefined) {
      user[field] = data[field];
    }
  });
  const emailAlreadyExist  = await User.findOne({email
    :user.email
  });

if(emailAlreadyExist && emailAlreadyExist._id.toString() !==
user._id.toString()){
  throw new AppError(400, 'Email already exists');
}

const NameAlreadyExist = await User.findOne({
  username: user.username
});

if (
  NameAlreadyExist &&
  NameAlreadyExist._id.toString() !== user._id.toString()
) {
  throw new AppError(400, "Username already exists");
```

```
}

 await user.save();

 return await User.findById(user._id).select('-password');
};

export const getAllUsers = async (query) => {
 const page = Number(query.page) || 1;
 const limit = Number(query.limit) || 10;

 const filter = { isActive: true };

 const totalItems = await User.countDocuments(filter);

 const pagination = new Pagination(User.find(filter), query)
   .filter()
   .sort()
   .limitFields()
   .paginate();

 const users = await pagination.query.select("-password").lean();

 const meta = {
   page,
   limit,
   totalItems,
   totalPages: Math.ceil(totalItems / limit),
   hasNextPage: page < Math.ceil(totalItems / limit),
 hasPreviousPage: page > 1,
 };

 const summary = {
   activeUsers: totalItems,
 };

 return {
   data: users,
   meta,
   summary,
```

```
 };
};
export const createUser = async (data) => {
 const newUser = await User.create(data);
 return await
User.findById(newUser._id).select('-password').lean().exec();
};

export const getUserById = async (id) => {
 return await User.findById(id).select('-password').lean().exec();
};

export const updateUserById = async (id, data) => {
 if(data.password || data.avatar ||data.avatarPublicId ){
   throw new AppError(400, 'password, avatar and avatarPublicId cannot be
updated');
 }
const emailAlreadyExist  = await User.findOne({email
   :data.email
 });

if(emailAlreadyExist && emailAlreadyExist._id.toString() !==
id.toString()){
 throw new AppError(400, 'Email already exists');
}

const NameAlreadyExist = await User.findOne({
 username: data.username
});

if (
 NameAlreadyExist &&
 NameAlreadyExist._id.toString() !== id.toString()
) {
 throw new AppError(400, "Username already exists");
}

 return await User.findByIdAndUpdate(id, data, { new: true, runValidators:
true }).select('-password').lean().exec();
};
```

```
export const deleteUserById = async (id) => {
   const user = await User.findById(id);
     await redis.del(`user:${id}:profile`);

   if (!user) {
       throw new AppError(404, "User not found");
   }

   if (user.avatarPublicId) {
       await imagekit.deleteFile(user.avatarPublicId);
       user.avatar = "";
       user.avatarPublicId = "";
   }

   user.isActive = false;

   await user.save();

   user.password = undefined;


   return user;
};
export const updateUserPassword = async (id, oldpassword, newPassword,
confirmNewPassword) => {
 const user = await User.findById(id).select('+password');
 if (!user) return false;

 const isMatch = await user.comparePassword(oldpassword);
 if (!isMatch) return false;

 if (newPassword !== confirmNewPassword) {
   return false;
 }

 user.password = newPassword;
  await user.save();
 return true;
};
```

Projects

```
const memberSchema = new mongoose.Schema(
 {
   user: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "User",
     required: [true, "Member user is required"],
   },
   role: {
     type: String,
     enum: projectRoles,
     default: "MEMBER",
   },
   joinedAt: {
     type: Date,
     default: Date.now,
   },
 },
 { _id: false }
);

const projectSchema = new mongoose.Schema(
 {
   name: {
     type: String,
     required: [true, "Project name is required"],
     trim: true,
     maxlength: [100, "Project name cannot exceed 100 characters"],
   },
   description: {
     type: String,
     default: "",
     maxlength: [2000, "Description cannot exceed 2000 characters"],
   },
   owner: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "User",
```

```
    required: [true, "Project owner is required"],
  },
  members: {
    type: [memberSchema],
    default: [],
  },

  color: {
    type: String,
    default: "#4f46e5",
    match: [/^#([0-9a-fA-F]{3}|[0-9a-fA-F]{6})$/, "Color must be a valid
hex value"],
  },
 },
 { timestamps: true }
);
```

```
const router = express.Router();
router.route('/').get(protect, getAllMyProjects).post(protect,
createProject);
router.route('/:projectId').get(protect,
authorizeProjectTo("OWNER","ADMIN","MEMBER"),getProjectById).patch(protect
, updateProject).delete(protect,
authorizeProjectTo("OWNER"),deleteProject);
router.delete('/:projectId/members/:userId', protect,
authorizeProjectTo("OWNER"),removeMember);
router.get('/:projectId/members/',protect,authorizeProjectTo("OWNER","ADMI
N","MEMBER"),getAllMembers);
router.patch('/:projectId/members/:userId/role', protect,
authorizeProjectTo("ADMIN","OWNER"), changeMemberRole);
// router.patch('/:projectId/archive', protect,
authorizeProjectTo("ADMIN","OWNER"),archiveProject);
// router.patch('/:projectId/restore', protect,
authorizeProjectTo("ADMIN","OWNER"),restoreProject);
router
 .route('/:projectId/tasks')
 .get(protect, authorizeProjectTo('OWNER', 'ADMIN', 'MEMBER'),
getTasksByProject)
```

```
 .post(protect, authorizeProjectTo('OWNER', 'ADMIN', 'MEMBER'),
createTask);


export const getAllMyProjects = catchAsync(async (req, res, next) => {
 const result = await getAllMyProjectsService(req.query,req.user.id);
 if (!result.data || result.data.length === 0) {
   return next(new AppError(404, 'you dont have projects yet'));
 }
 return res.status(200).json(ApiResponse.success('here are your projects',
result.data, result.meta, result.summary));
});

export const createProject = catchAsync(async (req, res, next) => {
 const created = await createProjectService(req.body, req.user.id);
 if (!created) {
   return next(new AppError(400, 'problem in creating project'));
 }

 return res.status(200)
   .json(ApiResponse.success('project created  successfully', created));
});

export const getProjectById = catchAsync(async (req, res) => {
 const project = await getProjectByIdService(req.params.projectId,
req.user.id);
 return res.status(200).json(ApiResponse.success('project found',
project));
});

//FIXED
export const updateProject = catchAsync(async (req, res) => {
 const project = await updateProjectService(req.params.projectId,
req.body,req.user.id);
 return res.status(201).json(ApiResponse.success('project updated
successfully', null));
});

export const deleteProject = catchAsync(async (req, res) => {
 const project = await deleteProjectService(req.params.projectId);
```

```
 return res.status(204).json(ApiResponse.success('project deleted
successfully'));
});

export const addMember = catchAsync(async (req, res) => {
 const project = await addMemberService(req.params.projectId,
req.body,req.user.id);
 return res.status(200).json(ApiResponse.success('member added
successfully', project));
});

export const removeMember = catchAsync(async (req, res) => {
 const project = await removeMemberService(req.params.projectId,
req.params.userId,req.user.id);
 return res.status(200).json(ApiResponse.success('member removed
successfully', project));
});

export const changeMemberRole = catchAsync(async (req, res) => {
 const project = await changeMemberRoleService(req.params.projectId,
req.params.userId, req.body,req.user.id);
 return res.status(200).json(ApiResponse.success('member role updated
successfully', project));
});


export const getAllMembers = catchAsync(async (req,res,next)=>{
 const members = await getAllMembersService(req.params.projectId);
 if(!members){
return next(new AppError(400,"Sorry no members"));
 }
 return res.status(200).json(ApiResponse.success('members returned
successfully', members));
});
export const getAllMyProjectsService = async (query,userId) => {

const page  = new Pagination(
 Project.find({
   $or: [
     { owner: userId },
```

```
      { "members.user": userId }
    ]
  })
    .populate("owner")
    .populate("members.user")

  ,query).filter().limitFields().sort().paginate();

const projects = await page.query.lean();

if(!projects){
throw new AppError(400,"sorry no projects");
}


  const totalItems = projects.length;
  const pageNumber = query.page || 1;
  const limit = query.limit || 10;

  const meta = {
    page: pageNumber,
    limit,
    totalItems,
    totalPages: Math.ceil(totalItems / limit),
    hasNextPage: pageNumber < Math.ceil(totalItems / limit),
    hasPreviousPage: pageNumber > 1,
  };

  const summary = {
    projectCount: totalItems,
  };
return { data: projects, meta, summary };
};

export const createProjectService = async (data, userId) => {
          await redis.del(`user:${userId}:profile`);
   const project = await Project.create({
    ...data,
    owner: userId,
    members: [
```

```
    {
      user: userId,
      role: "OWNER"
    }
  ]
});

await Promise.all([
  ActivityLog.create({
      project: project._id,
      actor: userId,
      type: "PROJECT_CREATED",
      entityType: "Project",
      entityId: project._id,
      message: `Project "${project.name}" was created`
  }),
  Notification.create({
      receiver: userId,
      type: "project",
      title: "Project Created",
      message: `Project "${project.name}" was created successfully`
  })
]);

return project;
};

export const getProjectByIdService = async (projectId, userId) => {
const project = await Project.findById(projectId);
if (!project) {
  throw new AppError(404, 'project not found');
}

const isMember = project.members.some((member) => member.user.toString()
=== userId.toString()) || project.owner.toString() === userId.toString();
if (!isMember) {
  throw new AppError(403, 'you are not a member of this project');
}

return await populateProject(project);
```

```
};

export const updateProjectService = async (projectid, data,userId) => {
 const allowedFields = ['name', 'description', 'color', 'visibility',
'status'];
const project = await Project.findById(projectid);
if(!project){
   throw new AppError(403, 'project doesnt even eist');

}
 allowedFields.forEach((field) => {
   if (data[field] !== undefined) {
     project[field] = data[field];
   }
 });

 await project.save();
   await ActivityLog.create({
   project: project._id,
   actor: userId,
   type: "PROJECT_UPDATED",
   entityType: "Project",
   entityId: project._id,
   message: `Project "${project.name}" was updated`
 });

 return await populateProject(project);
};

export const deleteProjectService = async (projectId) => {
 const project = await Project.findById(projectId);

 if (!project) {
   throw new AppError(404, "Project not found");
 }

 const attachments = await Attachment.find({ project: projectId });

 await Promise.all(
   attachments.map(async (attachment) => {
```

```
    if (attachment.publicId) {
      await imagekit.deleteFile(attachment.publicId);
    }
  })
);

await Promise.all([
  Comment.deleteMany({ task: { $in: await Task.find({ project: projectId
}).distinct("_id") } }),
  Task.deleteMany({ project: projectId }),
  Attachment.deleteMany({ project: projectId }),
  Invitation.deleteMany({ project: projectId }),
  ActivityLog.deleteMany({ project: projectId }),
  Project.findByIdAndDelete(projectId),
]);

return project;
};

export const addMemberService = async (project, data,actorId) => {
const userId = data?.userId;
const role = data?.role?.toUpperCase();

if (!userId) {
  throw new AppError(400, 'userId is required');
}

if (project.owner.toString() === userId.toString()) {
  throw new AppError(400, 'owner is already a member of this project');
}

const alreadyMember = project.members.some((member) =>
member.user.toString() === userId.toString());
if (alreadyMember) {
  throw new AppError(400, 'member already exists');
}

const user = await User.findById(userId);
if (!user) {
  throw new AppError(404, 'user not found');
```

```
 }

 const validRole = role === 'ADMIN' ? 'ADMIN' : 'MEMBER';
 project.members.push({ user: userId, role: validRole });
await Promise.all([
 project.save(),
 ActivityLog.create({
   project: project._id,
   actor: actorId,
   type: "MEMBER_JOINED",
   entityType: "Project",
   entityId: project._id,
   message: `User: ${user.username} joined project "${project.name}"`
 })
]);
 return await populateProject(project);
};

export const removeMemberService = async (projectid, userId, actorId) => {

 const project = await Project.findById(projectid);
 if (!project) {
   throw new AppError(404, 'project not found');
 }
 if (project.owner.toString() === userId.toString()) {
   throw new AppError(400, 'owner cannot be removed');
 }

 const user = await User.findById(userId);
 if (!user) {
   throw new AppError(404, 'user not found');
 }
         await redis.del(`user:${userId}:profile`);

 const memberIndex = project.members.findIndex((member) =>
member.user.toString() === userId.toString());
 if (memberIndex === -1) {
   throw new AppError(404, 'member not found');
 }
```

```
 project.members.splice(memberIndex, 1);
 await Promise.all([
   project.save(),
   ActivityLog.create({
     project: project._id,
     actor: actorId,
     type: "MEMBER_LEFT",
     entityType: "Project",
     entityId: project._id,
     message: `User :${user.username} left project "${project.name}"`
   })
 ]);
 return await populateProject(project);
};

export const changeMemberRoleService = async (project, userId,
data,actorId) => {
 const role = data?.role?.toUpperCase();

 if (!role) {
   throw new AppError(400, 'role is required');
 }

 if (!['ADMIN', 'MEMBER'].includes(role)) {
   throw new AppError(400, 'role must be ADMIN or MEMBER');
 }
 const myproject = await Project.findById(project );
 console.log(myproject);
 const member = myproject.members.find((item) => item.user.toString() ===
userId.toString());
 if (!member) {
   throw new AppError(404, 'member not found');
 }

 if (member.role === 'OWNER') {
   throw new AppError(400, 'owner role cannot be changed');
 }

 member.role = role;
 await Promise.all([
```

```
myproject.save(),
 ActivityLog.create({
   project: myproject,
   actor: actorId,
   type: "MEMBER_ROLE_CHANGED",
   entityType: "Project",
   entityId: project,
   message: `User : ${member.username}role changed to ${role} in project
"${myproject.name}"`
 })
]);

 return await populateProject(myproject);
};

// export const archiveProjectService = async (project) => {
//   project.status = 'Archived';
//   await project.save();
//   return await populateProject(project);
// };



// export const restoreProjectService = async (project) => {
//   project.status = 'Active';
//   await project.save();
//   return await populateProject(project);
// };
export const getAllMembersService = async (projectId) => {
 // Chain the populate methods directly to the query
 const project = await Project.findById(projectId)
   .populate("owner")
   .populate("members.user");

 if (!project) {
   throw new AppError(404, "couldnt find project");
 }

 // Now that the project is populated, you can safely return the members
 return project.members;
};
```

Tasks

```
const taskSchema = new mongoose.Schema(
 {
   project: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "Project",
     required: [true, "Task must belong to a project"],
   },
   title: {
     type: String,
     required: [true, "Task title is required"],
     trim: true,
     maxlength: [120, "Task title cannot exceed 120 characters"],
   },
   description: {
     type: String,
     default: "",
     maxlength: [4000, "Description cannot exceed 4000 characters"],
   },
   status: {
     type: String,
     enum: ["TODO", "IN_PROGRESS", "DONE"],
     default: "TODO",
   },
   priority: {
     type: String,
     enum: ["LOW", "MEDIUM", "HIGH"],
     default: "MEDIUM",
   },
   assignee: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "User",
     default: null,
   },
   creator: {
     type: mongoose.Schema.Types.ObjectId,
```

```
    ref: "User",
    required: [true, "Task creator is required"],
  },
  dueDate: {
    type: Date,
    default: null,
    validate: {
      validator: function (value) {
        return !value || value >= new Date();
      },
      message: "Due date cannot be in the past",
    },
  },
  labels: {
    type: [String],
    default: [],
  },

  attachments: {
    type: [{ type: mongoose.Schema.Types.ObjectId, ref: "Attachment" }],
    default: [],
  },
  comments: {
    type: [{ type: mongoose.Schema.Types.ObjectId, ref: "Comment" }],
    default: [],
  },
  order: {
    type: Number,
    default: 0,
    min: [0, "Order cannot be negative"],
  },
 },
 { timestamps: true }
);

router.route("/").get(protect,getAllTasks)
router.route('/:taskId')
 .get(protect, authorizeTaskTo('OWNER', 'ADMIN','MEMBER'), getTaskById)
 .patch(protect, authorizeTaskTo('OWNER', 'ADMIN'), updateTask)
 .delete(protect, authorizeTaskTo('OWNER', 'ADMIN'), deleteTask);
```

```
router.route('/:taskId/comments').post(protect,authorizeTaskTo('OWNER',
'ADMIN','MEMBER'),upload.single("file") ,createComment);
//.get(protect,authorizeTaskTo('OWNER', 'ADMIN','MEMBER'),
getCommentsByTask);

router.route('/:taskId/attachments').post(protect,authorizeTaskTo('OWNER',
'ADMIN','MEMBER') ,upload.single('file'),createAttachment);
//.get(protect, authorizeTaskTo('OWNER',
'ADMIN','MEMBER'),getAttachmentsByTask);

router.delete("/:taskId/attachments/:attachmentId",protect,authorizeTaskTo
('OWNER', 'ADMIN','MEMBER') ,deleteAttachment)

export const getAllTasks  =catchAsync(async (req,res,next)=>{
 const result = await getMyTaskService(req.query,req.user.id);
 if(!result.data || result.data.length === 0){
   return next(new AppError(404, 'No tasks found for this project'));
 }
   return res.status(200).json(ApiResponse.success('Tasks retrieved
successfully', result.data,result.meta,result.summary));

});
export const getTasksByProject = catchAsync(async (req, res, next) => {
 const tasks = await
getTasksByProjectService(req.query,req.params.projectId);
 if (!tasks) {
   return next(new AppError(404, 'No tasks found for this project'));
 }
 return res.status(200).json(ApiResponse.success('Tasks retrieved
successfully', tasks));
});

export const createTask = catchAsync(async (req, res, next) => {
 const task = await createTaskService(req.params.projectId, req.body,
req.user.id);
 if (!task) {
   return next(new AppError(400, 'Could not create task'));
 }
```

```
 return res.status(201).json(ApiResponse.success('Task created
successfully', task));
});

export const getTaskById = catchAsync(async (req, res, next) => {
 const task = await getTaskByIdService(req.params.taskId);
 return res.status(200).json(ApiResponse.success('Task retrieved
successfully', task));
});

export const updateTask = catchAsync(async (req, res, next) => {
 const task = await updateTaskService(req.params.taskId, req.body);
 return res.status(200).json(ApiResponse.success('Task updated
successfully', task));
});

export const deleteTask = catchAsync(async (req, res, next) => {
 await deleteTaskService(req.params.taskId);
 return res.status(200).json(ApiResponse.success('Task deleted
successfully', null));
});



export const getMyTaskService = async (query,userId)=>{

const page = new Pagination(
 Task.find({
   $or: [
     { creator: userId },        // Condition 1: You created/own the task
     { assignee: userId }    // Condition 2: You are assigned to the task
   ]
 }),
 query
)
 .filter()
 .sort()
 .limitFields()
 .paginate();
```

```
// 3. Execute the query
const tasks = await page.query.lean();

return await Promise.all(tasks.map(populateTask));
}
export const getTasksByProjectService = async (query,projectId) => {
 const project = await Project.findById(projectId);


 if (!project) {
   throw new AppError(404, 'project not found');
 }

 const page = new Pagination(
Task.find({ project: projectId }).populate([{ path: 'creator' }, { path:
'assignee' }]),
 query
)
 .filter()
 .sort()
 .limitFields()
 .paginate();



const tasks = await page.query.lean();
const populatedTasks= await Promise.all(tasks.map(populateTask))

 const meta = {
   page:query.page || 1,
   limit:query.limit || 10,
   totalItems:populatedTasks.length,
   totalPages: Math.ceil(populatedTasks.length / query.limit),
   hasNextPage: query.page <Math.ceil(populatedTasks.length /
query.limit),
 hasPreviousPage: query.page > 1,
 };

 const summary = {
   taskCount: populatedTasks.length,
```

```
 };

return {
 data:populatedTasks,
 meta,
 summary
};
};

export const createTaskService = async (projectId, data, userId) => {
   await redis.del(`user:${userId}:profile`);

 const project = await Project.findById(projectId);
 if (!project) {
   throw new AppError(404, 'project not found');
 }

 let assignee = null;

 if (data.assignee !== undefined && data.assignee !== null) {
   assignee = await User.findById(data.assignee);
   if (!assignee) {
     throw new AppError(404, 'assignee not found');
   }

   const isMember = project.members.some(
     (member) => member.user.toString() === assignee._id.toString()
   );

   if (!isMember) {
     throw new AppError(404, 'sorry he isn\'t a member invite him first');
   }
       await redis.del(`user:${data.assignee}:profile`);

 }

 const validStatuses = ['TODO', 'IN_PROGRESS', 'DONE'];
 const validPriorities = ['LOW', 'MEDIUM', 'HIGH'];

 if (data.status && !validStatuses.includes(data.status)) {
```

```
    throw new AppError(400, 'invalid task status');
  }

  if (data.priority && !validPriorities.includes(data.priority)) {
    throw new AppError(400, 'invalid task priority');
  }

  const lastTask = await Task.findOne({ project: projectId }).sort({ order:
-1, createdAt: -1 }).select('order');

  const task = new Task({
    project: projectId,
    creator: userId,
    title: data.title,
    description: data.description || '',
    assignee: assignee ? assignee._id : null,
    status: data.status || 'TODO',
    priority: data.priority || 'MEDIUM',
    dueDate: data.dueDate || null,
    labels: normalizeLabels(data.labels),
    order: lastTask ? lastTask.order + 1 : 0,
  });

  await task.save();

  if (assignee) {
    await notificationsService.createNotification({
      receiver: assignee._id,
      sender: userId,
      type: 'task',
      title: 'Task assigned',
      message: `You have been assigned to "${task.title}" in project
"${project.name}".`,
      metadata: {
        projectId: project._id,
        taskId: task._id,
        projectName: project.name,
      },
    });
  }
```

```
 await ActivityLog.create({
   project: projectId,
   actor: userId,
   type: 'TASK_CREATED',
   entityType: 'Task',
   entityId: task._id,
   message: `Task "${task.title}" created by ${task.creator}`,
 })

 return await populateTask(task);
};

export const getTaskByIdService = async (taskId) => {
 const task = await Task.findById(taskId);
 if (!task) {
   throw new AppError(404, 'task not found');
 }

 return await populateTask(task);
};

export const updateTaskService = async (taskId, data) => {

 const task = await Task.findById(taskId).populate('project');
 if (!task) {
   throw new AppError(404, 'task not found');
 }

 const previousAssigneeId = task.assignee ? task.assignee.toString() :
null;
 const previousStatus = task.status;

 if (data.assignee !== undefined) {
   let assignee = null;

   if (data.assignee !== null) {
     assignee = await User.findOne({email:data.assignee});
     if (!assignee) {
       throw new AppError(404, 'assignee not found');
```

```
    }

    const isMember = task.project.members.some(
      (member) => member.user.toString() === assignee._id.toString()
    );

    if (!isMember) {
      throw new AppError(404, 'sorry he isn\'t a member invite him
first');
    }
  }
  console.log(task.assignee);
  task.assignee = assignee ? assignee._id : null;
      await redis.del(`user:${task.creator}:profile`);
  await redis.del(`user:${task.assignee}:profile`);

}

const validStatuses = ['TODO', 'IN_PROGRESS', 'DONE'];
const validPriorities = ['LOW', 'MEDIUM', 'HIGH'];

if (data.status !== undefined) {
  if (!validStatuses.includes(data.status)) {
    throw new AppError(400, 'invalid task status');
  }
  task.status = data.status;

}

if (data.priority !== undefined) {
  if (!validPriorities.includes(data.priority)) {
    throw new AppError(400, 'invalid task priority');
  }
  task.priority = data.priority;
}

if (data.title !== undefined) {
  task.title = data.title;
}
```

```
 if (data.description !== undefined) {
   task.description = data.description;
 }

 if (data.dueDate !== undefined) {
   task.dueDate = data.dueDate;
 }

 if (data.labels !== undefined) {
   task.labels = normalizeLabels(data.labels);
 }

 await task.save();
if(task.status !== previousStatus){
await ActivityLog.create({
   project: task.project._id,
   actor: task.creator,
   type: 'TASK_STATUS_CHANGED',
   entityType: 'Task',
   entityId: task._id,
   message: `Task "${task.title}" ${previousStatus} -> ${task.status} by
${task.creator}`,
 });
 }
 const shouldNotifyForAssignment = Boolean(
   task.assignee && previousAssigneeId !== task.assignee.toString()
 );

 if (shouldNotifyForAssignment) {
   await notificationsService.createNotification({
     receiver: task.assignee,
     sender: task.creator,
     type: 'task',
     title: 'Task assigned',
     message: `You have been assigned to "${task.title}" in project
"${task.project.name}".`,
     metadata: {
       projectId: task.project._id,
       taskId: task._id,
       projectName: task.project.name,
```

```
    },
   });
 }
  await ActivityLog.create({
   project: task.project._id,
   actor: task.creator,
   type: 'TASK_UPDATED',
   entityType: 'Task',
   entityId: task._id,
   message: `Task "${task.title}" updated by ${task.creator}`,
 });
 return await populateTask(task);
};

export const deleteTaskService = async (taskId) => {
 const task = await Task.findById(taskId);
 if (!task) {
   throw new AppError(404, 'task not found');
 }

 const attachments = await Attachment.find({ task: taskId });
  for (const attachment of attachments) {
     await imagekit.deleteFile(attachment.publicId);
  }

Promise.all([
redis.del(`user:${task.creator}:profile`),Attachment.deleteMany({ task:
taskId }), Comment.deleteMany({ task: taskId }),
Task.findByIdAndDelete(taskId)]);
 return task;
};
```

Notifications

```
const notificationSchema = new mongoose.Schema(
 {
   receiver: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "User",
```

```
      required: [true, "Notification receiver is required"],
    },
    sender: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      default: null,
    },
    type: {
      type: String,
      enum: ["invitation", "task", "comment", "message", "project",
"system"],
      required: [true, "Notification type is required"],
    },
    title: {
      type: String,
      required: [true, "Notification title is required"],
      trim: true,
      maxlength: [120, "Notification title cannot exceed 120 characters"],
    },
    message: {
      type: String,
      required: [true, "Notification message is required"],
      trim: true,
      maxlength: [2000, "Notification message cannot exceed 2000
characters"],
    },
    read: {
      type: Boolean,
      default: false,
    },
    metadata: {
      type: mongoose.Schema.Types.Mixed,
      default: {},
    },
  },
  { timestamps: true }
);

router.get('/', protect, notificationsController.getNotifications);
router.patch('/read', protect, notificationsController.markAsRead);
```

```
export const notificationsController = {
 getNotifications: catchAsync(async (req, res,next) => {
   const result = await
notificationsService.getUserNotifications(req.query,req.user.id);
   if(result.data.length === 0){
return next(new AppError(404,"sorry couldnt find notificaions"))
   }

   res.status(200).json(ApiResponse.success('Notifications retrieved
successfully', result.data, result.meta, result.summary));
 }),

 markAsRead: catchAsync(async (req, res) => {
   const notification = await
notificationsService.markAllAsRead(req.user.id);

   if (!notification) {
     return res.status(404).json(ApiResponse.fail('Notification not
found'));
   }

   return res.status(200).json(ApiResponse.success('Notification marked as
read', notification));
 }),


};

export const notificationsService = {
 createNotification: async ({ receiver, sender = null, type, title,
message, metadata = {} }) => {
   if (!receiver) {
     return null;
   }

   return Notification.create({
     receiver,
     sender,
     type,
```

```
    title,
    message,
    metadata,
  });
},

getUserNotifications: async (query,userId) => {
  const page = new Pagination(Notification.find({ receiver: userId
}).sort({ createdAt: -1 })
  ,query).filter().limitFields().paginate().sort();

  const results = await page.query.lean();


const meta = {
  page: query.page || 1,
  limit: query.limit || 10,
  totalItems: results.length,
  totalPages: Math.ceil(results.length / (query.limit || 10)),
  hasNextPage: (query.page || 1) < Math.ceil(results.length /
(query.limit || 10)),
  hasPreviousPage: (query.page || 1) > 1,
};

const summary = {
  notificationsCount: results.length,
};
  return { data: results, meta, summary };
},

markAllAsRead: async (userId) => {
  await redis.del(`user:${userId}:profile`);

  await Notification.updateMany(
      {
          receiver: userId,
          read: false
      },
      {
          $set: {
```

```
                read: true
            }
        }
    );

    return;
},
};
```

Project Logs

```
const activityTypes = [
 "PROJECT_CREATED",
 "PROJECT_UPDATED",


 "TASK_CREATED",
 "TASK_UPDATED",
 "TASK_DELETED",
 "TASK_STATUS_CHANGED",

 "COMMENT_CREATED",
 "COMMENT_DELETED",
 "MEMBER_JOINED",
 "MEMBER_LEFT",
 "MEMBER_ROLE_CHANGED",

 "INVITATION_SENT",
 "INVITATION_ACCEPTED",
 "INVITATION_REJECTED",

 "ATTACHMENT_UPLOADED",
 "ATTACHMENT_DELETED"
];

const activityLogSchema = new mongoose.Schema(
 {
   project: {
     type: mongoose.Schema.Types.ObjectId,
```

```
    ref: "Project",
    required: true,
    index: true,
  },

  actor: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },

  type: {
    type: String,
    enum: activityTypes,
    required: true,
  },

  entityType: {
    type: String,
    enum: [
      "Project",
      "Task",
      "Comment",
      "Invitation",
      "Attachment",
      "Member",
    ],
    required: true,
  },

  entityId: {
    type: mongoose.Schema.Types.ObjectId,
  },

  message: {
    type: String,
    required: true,
    maxlength: 300,
  },
```

```
   createdAt:{
     type: Date,
     default: Date.now,
     index: true,
   }
 },
 {
   timestamps: true,
 }
);

activityLogSchema.index({ project: 1, createdAt: -1 });
activityLogSchema.index({ actor: 1 });

export const ActivityLog = mongoose.model(
 "ActivityLog",
 activityLogSchema
);
```

```
router.get('/:projectId/activity',protect,authorizeProjectTo("OWNER","ADMI
N","MEMBER"),projectActivity);
export const projectActivity = catchAsync(async (req,res,next)=>{
 const activity = await getAllActivitesService(req.params.projectId);
 if(!activity){
return next(new AppError(400,"Sorry no activities"));
 }
 return res.status(200).json(ApiResponse.success('activites returned
successfully', activity));
});
export const getAllActivitesService = async (projectId) => {
try{
 const activity = await ActivityLog.find({ project: projectId }).sort({
createdAt: -1 });
 return activity;
}catch(err){
     throw new AppError(404, "couldnt find project");


}
```

```
}
```

Inviations

```
const invitationSchema = new mongoose.Schema(
 {
   project: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "Project",
     required: [true, "Invitation must target a project"],
   },
   sender: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "User",
     required: [true, "Sender is required"],
   },
   receiver: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "User",
     required: [true, "Receiver is required"],
   },
   status: {
     type: String,
     enum: ["Pending", "Accepted", "Rejected"],
     default: "Pending",
   },
   expiresAt: {
     type: Date,
     required: [true, "Invitation expiration date is required"],
     default: () => new Date(Date.now() + 7 * 24 * 60 * 60 * 1000),
   },
 },
 { timestamps: true }
);
```

```
router.post('/:projectId', protect, authorizeProjectTo('ADMIN', 'OWNER'),
createInvitation);
```

```
router.patch('/:projectId/:invitationId', protect,
authorizeProjectTo('ADMIN', 'OWNER'), editInvitation);
router.delete('/:projectId/:invitationId', protect,
authorizeProjectTo('ADMIN', 'OWNER'), deleteInvitation);
router.patch('/:invitationId', protect, reactToinvitation);
router.get('/', protect, myInvitations)


export const createInvitation = catchAsync(async (req, res, next) => {
 const { recieverEmail } = req.body;
 const projectId = req.params.projectId;
 if (!recieverEmail || !projectId) {
   return next(new AppError(400, 'sorry missing fieleds'));
 }

 const invitation = await sendInvitation(req.user.id, recieverEmail,
projectId);
 if (!invitation) {
   return next(new AppError(400, 'sorry could send invitation'));
 }

  res.status(200).json(ApiResponse.success('inviation made and sent',
invitation));
});

export const editInvitation = catchAsync(async (req, res, next) => {
 const { projectId, invitationId } = req.params;
 const invitation = await editInvitationService(projectId, invitationId,
req.body,req.user.id);

 if (!invitation) {
   return next(new AppError(400, 'sorry could not update invitation'));
 }

  res.status(200).json(ApiResponse.success('invitation updated
successfully', invitation));
});

export const deleteInvitation = catchAsync(async (req, res, next) => {
 const { projectId, invitationId } = req.params;
```

```
 const invitation = await deleteInvitationService(projectId,
invitationId,req.user.id);

 if (!invitation) {
   return next(new AppError(400, 'sorry could not delete invitation'));
 }

  res.status(204).json(ApiResponse.success('invitation deleted
successfully'));
});

export const myInvitations = catchAsync(async (req, res,next) => {
 const userId = req.user.id;
 const result = await getMyInvitations(userId,req.query);
 if(result.data.length === 0){
return res.status(200).json(ApiResponse.success('No invitations
found',result.data,result.meta,result.summary));
 }
  res.status(200).json(
   ApiResponse.success('Invitations retrieved
successfully',result.data,result.meta,result.summary)
 );
});
export const reactToinvitation = catchAsync(async (req,res,next)=>{

 const {status}= req.body;
 const invitationId = req.params.invitationId;
 const reaction = await
reactToinvitationService(req.user.id,invitationId,status)
 res.status(200).json(ApiResponse.success("yiu have succesfully reacted to
the invitaation",reaction));
});
export const sendInvitation = async (userId, recieverEmail, projectId) =>
{
 const reciever = await User.findOne({ email: recieverEmail, isActive:
true });
 if (!reciever) {
   throw new AppError(404, 'sorry reciever email doesnt exist');
 }
```

```
 const project = await Project.findById(projectId);
 if (!project) {
   throw new AppError(404, 'project not found');
 }

 const isAlreadyMember = project.members.some(
   (member) => member.user.toString() === reciever._id.toString()
 );

 if (isAlreadyMember) {
   throw new AppError(400, 'cant invite a Member');
 }

 const invitation = await Invitation.create({
   project: projectId,
   receiver: reciever._id,
   sender: userId,
 });

await Promise.all([ActivityLog.create({
   project: project._id,
   actor: userId,
   type: "INVITATION_SENT",
   entityType: "Project",
   entityId: project._id,
   message: ` ${reciever.username} was invited to join the project
"${project.name}".`,
 }), notificationsService.createNotification({
   receiver: reciever._id,
   sender: userId,
   type: 'invitation',
   title: 'Project invitation',
   message: `You have been invited to join the project
"${project.name}".`,
   metadata: {
     projectId: project._id,
     projectName: project.name,
     invitationId: invitation._id,
   },
 })]);
```

```
 return invitation;
};

export const editInvitationService = async (projectId, invitationId,
data,userId) => {
 const invitation = await Invitation.findOne({ _id: invitationId, project:
projectId });
 await redis.del(`user:${userId}:profile`);

 if (!invitation) {
   throw new AppError(404, 'invitation not found');
 }

 // Prevent modifying an invitation that was already handled
 if (invitation.status === 'Accepted' || invitation.status === 'Rejected')
{
   throw new AppError(400, 'This invitation has already been handled');
 }

 if (data.status) {
   const validStatuses = ['Pending', 'Accepted', 'Rejected'];
   if (!validStatuses.includes(data.status)) {
     throw new AppError(400, 'invalid invitation status');
   }
   invitation.status = data.status;

   // FIX: If accepted, automatically add the user to the project's
members array
   if (data.status === 'Accepted') {
     const project = await Project.findById(projectId);
     if (!project) {
       throw new AppError(404, 'Associated project not found');
     }

     // Check if they are already a member to prevent duplicates
     const isAlreadyMember = project.members.some(
       (m) => m.user.toString() === invitation.receiver.toString()
     );

     if (!isAlreadyMember) {
```

```
      project.members.push({ user: invitation.receiver, role: 'MEMBER'
});
      await project.save(); // This triggers your pre('save') hook
safely!
    }
  }
 }

 if (data.expiresAt) {
   invitation.expiresAt = new Date(data.expiresAt);
 }

 await invitation.save();
 return invitation;
};

export const deleteInvitationService = async (projectId,
invitationId,userId) => {
 const invitation = await Invitation.findOneAndDelete({ _id: invitationId,
project: projectId });
 await redis.del(`user:${userId}:profile`);

 if (!invitation) {
   throw new AppError(404, 'invitation not found');
 }

 return invitation;
};

export const getMyInvitations = async (userId, query) => {
 const page = new Pagination(
   Invitation.find({
     $or: [
       { receiver: userId },
       { sender: userId }
     ]
   }).populate('project', 'name'),
   query
 )
   .filter()
```

```
   .sort() // Automatically sorts by newest first
   .limitFields()
   .paginate();

 // 2. Execute and return as a single paginated array
const invitations = await page.query.lean();


 const meta = {
   page: query.page || 1,
   limit: query.limit || 10,
   totalItems: invitations.length,
   totalPages: Math.ceil(invitations.length / (query.limit || 10)),
   hasNextPage: (query.page || 1) < Math.ceil(invitations.length /
(query.limit || 10)),
   hasPreviousPage: (query.page || 1) > 1,
 };

 const summary = {
   invitations: invitations.length,
 };
 return { data: invitations, meta: meta, summary: summary };
};


export const reactToinvitationService = async (userId, invitationId,
status) => {
 await redis.del(`user:${userId}:profile`);
 const invitation = await Invitation.findById(invitationId)
   .populate('project')
   .populate('receiver', 'email');

 if (!invitation) {
   throw new AppError(404, 'Invitation not found');
 }

 if (!invitation.project) {
   throw new AppError(404, 'The project associated with this invitation no
longer exists');
 }
```

```
 if (invitation.receiver._id.toString() !== userId.toString()) {
   throw new AppError(403, 'You are not authorized to respond to this
invitation');
 }

 if (invitation.status !== 'Pending') {
   throw new AppError(400, 'This invitation has already been handled');
 }

 const project = invitation.project;
 const sender = invitation.sender;
 const receiverEmail = invitation.receiver.email;
 let loggType ,  has;
 if (status === 'Accepted') {

   const isAlreadyMember = project.members.some(
     (member) => member.user.toString() === userId.toString()
   );

   if (!isAlreadyMember) {
     project.members.push({
       user: userId,
       role: 'MEMBER',
     });

     await project.save();
   }

   invitation.status = 'Accepted';
   loggType = "INVITATION_ACCEPTED"
   has  = "accepted";

 } else if (status === 'Rejected') {

   invitation.status = 'Rejected';
   loggType = "INVITATION_REJECTED";
   has  = "rejected";
 } else {
```

```
   throw new AppError(400, 'Invalid status action. Use Accepted or
Rejected.');
 }

 await Promise.all([invitation.save(),
   notificationsService.createNotification({
   receiver: sender,
   sender: userId,
   type: 'invitation reaction',
   title: 'Project invitation',
   message: `${receiverEmail} has ${invitation.status.toLowerCase()} your
invitation to "${project.name}"`,
   metadata: {
     projectId: project._id,
     projectName: project.name,
     invitationId: invitation._id,
   },
 }),ActivityLog.create({
   project:project,
   actor:receiverEmail,
   type: loggType,
   entityType:"Invitation",
   entityId:invitation._id,
   message:`${receiverEmail} has ${has} invitation`
 })]);
 return invitation;
};
```

Comments

```
const commentSchema = new mongoose.Schema(
 {
   task: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "Task",
     required: [true, "Comment must belong to a task"],
   },
   author: {
```

```
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: [true, "Comment author is required"],
  },
  content: {
    type: String,
    required: [true, "Comment content is required"],
    trim: true,
    maxlength: [4000, "Comment content cannot exceed 4000 characters"],
  },
  edited: {
    type: Boolean,
    default: false,
  },
  editedAt: {
    type: Date,
    default: null,
  },
      commentAttachments: {
        type: [{ type: mongoose.Schema.Types.ObjectId, ref: "Attachment"
}],
        default: [],
      },
 },
 { timestamps: true }
);

router.route('/:commentId').delete(protect, deleteComment);

export const getCommentsByTask = catchAsync(async (req, res, next) => {
 const comments = await getCommentsByTaskService(req.params.taskId);
 return res.status(200).json(ApiResponse.success('Comments retrieved
successfully', comments));
});

export const createComment = catchAsync(async (req, res, next) => {
 const comment = await createCommentService(req.params.taskId, req.body,
req.user.id,req.file);
 if (!comment) {
   return next(new AppError(400, 'Could not create comment'));
```

```
 }
 return res.status(201).json(ApiResponse.success('Comment created
successfully', comment));
});

// export const updateComment = catchAsync(async (req, res, next) => {
//   const comment = await updateCommentService(req.params.commentId,
req.body,req.user.id, req.file);
//   return res.status(200).json(ApiResponse.success('Comment updated
successfully', comment));
// });

export const deleteComment = catchAsync(async (req, res, next) => {
 await deleteCommentService(req.params.commentId,req.user.id);
 return res.status(200).json(ApiResponse.success('Comment deleted
successfully', null));
});

xport const getCommentsByTaskService = async (taskId) => {
 const task = await Task.findById(taskId);
 if (!task) {
   throw new AppError(404, 'task not found');
 }

 return await Comment.find({ task: taskId }).populate('author');
};

export const createCommentService = async (taskId, data, userId,file) => {
 const task = await Task.findById(taskId);
 if (!task) {
   throw new AppError(404, 'task not found');
 }
 if (!data.content) {
   throw new AppError(400, 'comment content is required');
 }
 const comment = new Comment({
   task: taskId,
   author: userId,
   content: data.content,
 });
```

```
if(file){
 try{
 const uploaded = await uploadToCloudinary(file);
   const attachment = new Attachment({
     uploadedBy: userId,
     project: task.project,
     task: taskId,
     fileName: file.originalname,
     originalName: file.originalname,
     mimeType: file.mimetype,
     size: file.size,
     url: uploaded.secure_url,
     publicId: uploaded.public_id,
   });
   await attachment.save();
   comment.commentAttachments.push(attachment._id);
}catch(err){
 throw new AppError(500,`file upload failed: ${err.message}`);
}



 }


   await Promise.all([
comment.save(),
ActivityLog.create({
   project: task.project,
   actor: userId,
   type: "COMMENT_CREATED",
   entityType: "Comment",
   entityId: comment._id,
   message: `Comment  was created on task "${task.title}"`,
 })
   ]);
    task.comments.push(comment._id);
 await task.save();
 return  populateComment(comment);
};
```

```
// export const updateCommentService = async (commentId, data,userId,file)
=> {
//   const comment = await Comment.findById(commentId);
//   if(userId.toString()!==comment.author.toString()){
//     throw new AppError(403, 'not your comment');

//   }
//   if (!comment) {
//     throw new AppError(404, 'comment not found');
//   }

//   if (!data.content) {
//     throw new AppError(400, 'comment content is required');
//   }

//   comment.content = data.content;
//   comment.edited = true;
//   comment.editedAt = new Date();

//   await comment.save();
//   return await populateComment(comment);
// };

export const deleteCommentService = async (commentId, userId) => {
 const comment = await Comment.findById(commentId);
 const task = await Task.findById(comment.task).lean();
 if (!comment) {
   throw new AppError(404, "Comment not found");
 }


 if (comment.author.toString() !== userId.toString()) {
   throw new AppError(403, "You can only delete your own comment");
 }
const attachments = await Attachment.find({ comment: commentId });

for (const attachment of attachments) {
   await imagekit.deleteFile(attachment.publicId);

}
```

```
await Promise.all([ Attachment.deleteMany({ comment: commentId
}),comment.deleteOne(),
 ActivityLog.create({
   project: task.project,
   actor: userId,
   type: "COMMENT_DELETED",
   entityType: "Comment",
   entityId: comment._id,
   message: `Comment  was deleted from task "${task.title}"`,
 })
]);


 return comment;
};
```

Attachments

```
const attachmentSchema = new mongoose.Schema(
 {
   uploadedBy: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "User",
     required: [true, "Uploader is required"],
   },
   project: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "Project",
     required: [true, "Project reference is required"],
   },
   task: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "Task",
     default: null,
   },
   fileName: {
     type: String,
     required: [true, "File name is required"],
     trim: true,
```

```
    },
    originalName: {
      type: String,
      required: [true, "Original file name is required"],
      trim: true,
    },
    publicId: {
      type: String,
      required: [true, "Cloud storage public ID is required"],
      trim: true,
    },
    url: {
      type: String,
      required: [true, "Attachment URL is required"],
      trim: true,
    },
    mimeType: {
      type: String,
      required: [true, "MIME type is required"],
      trim: true,
      match: [/^[a-z]+\/[a-z0-9.+-]+$/i, "Please provide a valid MIME
type"],
    },
    size: {
      type: Number,
      required: [true, "File size is required"],
      min: [1, "File size must be greater than zero"],
    },
  },
  { timestamps: true }
);
export const getAttachmentsByTask = catchAsync(async (req, res, next) => {
 const attachments = await getAttachmentsByTaskService(req.params.taskId);
 return res.status(200).json(ApiResponse.success('Attachments retrieved
successfully', attachments));
});

export const createAttachment = catchAsync(async (req, res, next) => {
 const attachment = await createAttachmentService(req.params.taskId,
req.file, req.user.id);
```

```
 if (!attachment) {
   return next(new AppError(400, 'Could not store attachment'));
 }
 return res.status(201).json(ApiResponse.success('Attachment stored
successfully', attachment));
});

export const deleteAttachment = catchAsync(async (req, res, next) => {
 await deleteAttachmentService(req.params.attachmentId);
 return res.status(200).json(ApiResponse.success('Attachment deleted
successfully', null));
});

export const getAttachmentsByTaskService = async (taskId) => {
 const task = await Task.findById(taskId);
 if (!task) {
   throw new AppError(404, 'task not found');
 }

 return await Attachment.find({ task: taskId }).populate('uploadedBy');
};
export const createAttachmentService = async (taskId, file, userId) => {
 if (!file) {
   throw new AppError(400, "file is required");
 }

 const task = await Task.findById(taskId);

 if (!task) {
   throw new AppError(404, "task not found");
 }

 let attachment;

 try {
   const uploaded = await uploadToCloudinary(file);

   const fileName = file.filename || file.originalname;

   attachment = new Attachment({
```

```
      uploadedBy: userId,
      project: task.project,
      task: taskId,
      fileName,
      originalName: file.originalname || fileName,
      mimeType: file.mimetype || uploaded.resource_type,
      size: uploaded.bytes,
      url: uploaded.secure_url,
      publicId: uploaded.public_id,
    });

    await attachment.save();

    task.attachments.push(attachment._id);
  } catch (err) {
    console.error(err);
    throw new AppError(500, "File upload failed");
  }

  await Promise.all([
    task.save(),
    ActivityLog.create({
      project: task.project,
      actor: userId,
      type: "ATTACHMENT_UPLOADED",
      entityType: "Attachment",
      entityId: attachment._id,
      message: `Attachment "${attachment.originalName}" has been uploaded
to task "${task.title}".`,
    }),
  ]);

  return await populateAttachment(attachment);
};
export const deleteAttachmentService = async (attachmentId) => {
  const attachment = await Attachment.findById(attachmentId);

  if (!attachment) {
    throw new AppError(404, 'attachment not found');
  }
```

```
 await Promise.all([imagekit.deleteFile(attachment.publicId),

Task.findByIdAndUpdate(
   attachment.task,
   {
     $pull: {
       attachments: attachment._id,
     },
   }
 ),
 attachment.deleteOne(),
ActivityLog.create({
   project: attachment.project,
   actor: attachment.uploadedBy,
   type: "ATTACHMENT_DELETED",
   entityType: "Attachment",
   entityId: attachment._id,
   message: `Attachment ${attachment.originalName} was deleted from task
${attachment.task}.`,
 })
]);

 return attachment;
};
```

Navigation Flow

Authentication

Login

- Success → Dashboard
- Register → Register

- Forgot Password → Forgot Password

Register

- Success → Dashboard
- Already have an account → Login

Forgot Password

- Success → Show "Check your email"
- Back → Login

Reset Password

- Success → Login

Dashboard

Navigation

- Projects
- Notifications
- Profile
- Logout

Projects

Navigation

- Dashboard
- Project Details
- Create Project

Project Details

Navigation

- Back to Projects
- Members
- Invitations
- Tasks

Profile

Navigation

- Dashboard
- Change Password

Notifications

Navigation

- Dashboard
- Open Project
- Open Task

User Roles

Owner

Can

- Edit Project
- Delete Project
- Invite Members
- Remove Members
- Change Member Roles
- Create Tasks
- Update Tasks
- Delete Tasks
- Upload Attachments
- Comment

Admin

Can

- Invite Members
- Change Member Roles
- Create Tasks
- Update Tasks
- Delete Tasks
- Upload Attachments
- Comment

Cannot

- Delete Project
- Remove Owner

Member

Can

- View Project
- Create Tasks
- Comment
- Upload Attachments

Cannot

- Delete Project

- Change Roles
- Remove Members

Loading States

Every page should support

Loading

- Skeleton UI

Empty

- Friendly empty message

Error

- Error banner
- Retry button

Responsive Design

Desktop

- Sidebar visible
- Four dashboard cards

- Kanban board with three columns

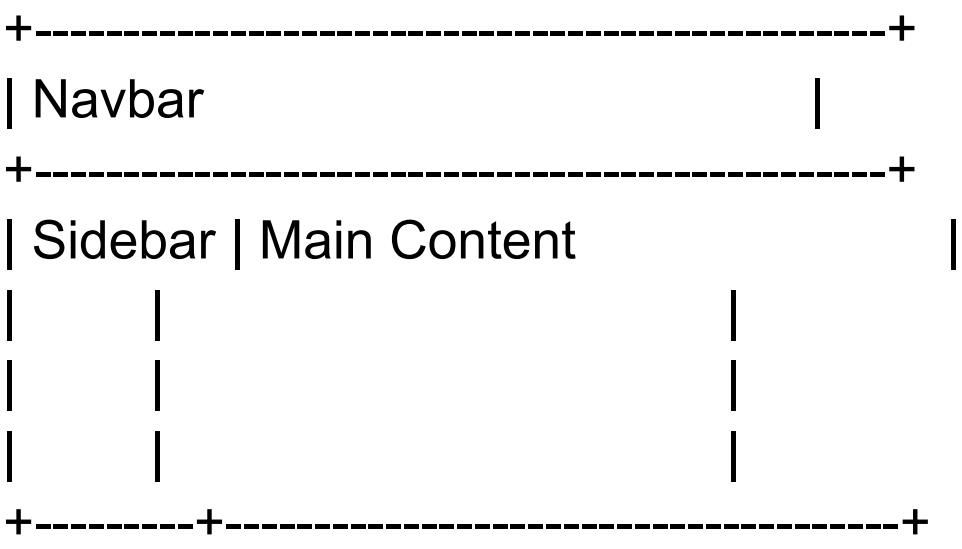
Tablet

- Sidebar collapses
- Two cards per row

Mobile

- Hamburger menu
- Cards stacked vertically
- Kanban scrolls horizontally

Global Layout



Navbar

Contains

- Logo
- Notifications
- Profile Avatar
- Logout

Sidebar

Contains

- Dashboard
- Projects
- Notifications
- Profile

Dashboard Layout

Navbar

Sidebar

Statistics Cards

Projects
Tasks
Completed
Notifications

Recent Projects

Recent Activity

Project Details

Navbar

Sidebar

Project Header

- Project Name
- Description
- Project Status
- Members Count
- Tasks Count
- Progress Bar

Members

Kanban Board

TODO

IN PROGRESS

DONE

Activity Feed (Project Logs)

Displays a chronological history of all project activity.

Each log entry displays:
- User Avatar
- Username
- Action performed
- Related entity (Task, Comment, Attachment, Invitation, Member)
- Timestamp

Examples:
- John created task "Authentication"
- Sarah changed task status to Done
- Ahmed uploaded "design.pdf"
- Mohamed commented on "Implement Dashboard"
- Ali invited Omar to the project
- Omar accepted the invitation
- Fatma removed a member from the project

Task Card

Displays

- Task Title
- Priority Badge
- Status
- Assignee Avatar
- Due Date
- Attachment Count
- Comment Count

Notification Card

Displays

- Icon
- Message
- Time
- Read Status

Unread

- Blue Dot

Read

- Gray

Project Card

Displays

- Project Name
- Description
- Members Count
- Tasks Count
- Color
- Progress Bar

Modal Windows

Project

- Create
- Edit
- Delete Confirmation

Task

- Create

- Edit
- Delete Confirmation

Invitation

- Send Invitation

Attachment

- Upload

Colors

Priority

Low → Green

Medium → Orange

High → Red

Task Status

Todo → Gray

In Progress → Blue

Done → Green

Notifications

Unread → Blue

Read → Gray

Danger Buttons

Delete → Red

Success

Create / Save → Green

Page State Flow

Every page should implement

Loading
↓
Success
↓
Empty State (if no data)

or

Loading
↓

Error
↓
Retry

API Usage Template

For every page, keep this structure:

Page Name

Purpose

Route

Access
- Guest
- User
- Admin

Layout

Components

APIs Used

Request Body

Success Response

Possible Errors

Navigation

Loading State

Empty State

Permissions

Notes